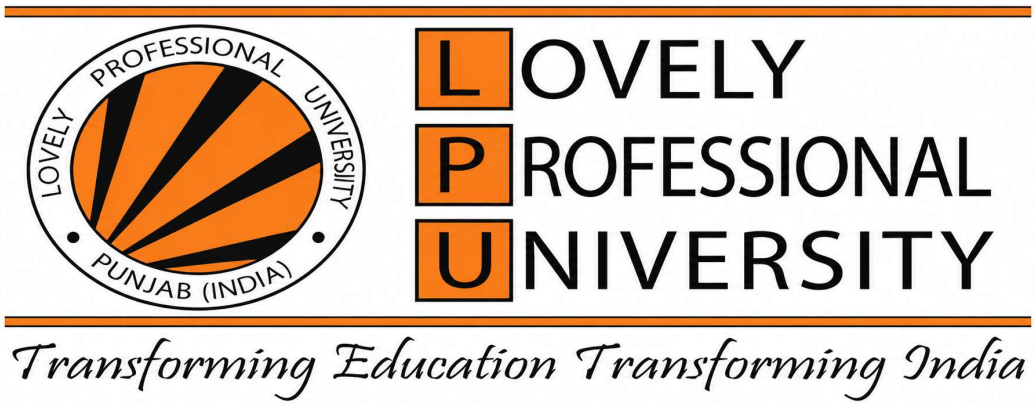
PROJECT REPORT



FOOD DELIVERY TIME PREDICTION SYSTEM

An Intelligent NoSQL Database Application Using MongoDB, PyMongo and Machine Learning

Subject	Intelligent NoSQL
Submitted by	Maria Tejaswi
Registration No:	12309910
Section:	K4P23DP
Roll No:	20
Submitted to	Dr. Ashu

Table of Contents

Section	Page
Abstract	4
1. Introduction	4
2. Problem Statement	4
3. Objectives	4
4. Technologies Used	5
5. Dataset Description	5
6. Intelligent NoSQL Database Design	6
6.1 Database and collections	5
6.2 Document-oriented data model	6
6.3 PyMongo operations	6
7. System Architecture and Workflow	7
8. Machine Learning Methodology	7
8.1 Target and input features	7
8.2 Preprocessing	7
8.3 Model selection	7

8.4 Evaluation metrics	8
9. Streamlit Application	8
10. NoSQL Concepts Demonstrated	8
11. Testing and Results	8
12. Limitations	9
13. Future Enhancements	9
14. Conclusion	9
References	10
Appendix A: Project Files	10
Appendix B: Submission Notes	10

Abstract

The Food Delivery Time Prediction System estimates the time required to deliver a food order using order distance, preparation time, weather, traffic level, time of day, vehicle type and courier experience. The project is designed for the Intelligent NoSQL subject, so MongoDB is a central part of the implementation rather than merely an optional storage layer. The delivery dataset is loaded into a MongoDB collection and accessed through PyMongo. A data-preprocessing pipeline handles missing values and converts categorical features into a model-compatible representation. A Random Forest Regression model is trained to estimate delivery time, and a Streamlit interface allows users to enter order details and view predictions. The app also stores prediction history in MongoDB, demonstrating document-oriented data storage and application-level database integration. In the recorded experiment, the model achieved a Mean Absolute Error (MAE) of 6.85 minutes and an R-squared score of 0.791. These results indicate useful predictive performance on the test split, although accuracy may vary on new or real-world delivery data.

Keywords: Intelligent NoSQL, MongoDB, PyMongo, document database, machine learning, Random Forest, Streamlit.

1. Introduction

Food delivery platforms need to provide customers with realistic delivery-time estimates. Delivery time can depend on several factors, including travel distance, food preparation time, traffic, weather, the time of day, the delivery vehicle and courier experience. A data-driven model can learn relationships between these factors and observed delivery times.

This project combines a NoSQL database with a supervised machine-learning model. MongoDB stores delivery records as flexible BSON documents, while PyMongo provides the Python interface used to insert and retrieve records. The trained model is exposed through a small Streamlit web application. Each new prediction can be saved as a document in a separate MongoDB collection, creating a simple prediction-history feature.

2. Problem Statement

To develop a simple application that predicts food delivery time from order and delivery conditions, while demonstrating how a document-oriented NoSQL database can store, retrieve and support application data. The solution should provide a usable interface and persist prediction history without requiring a relational schema.

3. Objectives

- Use MongoDB Atlas as the document database for delivery records and prediction history.
- Connect Python to MongoDB using PyMongo.
- Inspect and preprocess a delivery-time dataset, including handling missing values.
- Train a Random Forest Regression model to estimate delivery duration in minutes.
- Evaluate the model using Mean Absolute Error and R-squared.

- Build a Streamlit interface that accepts delivery inputs and displays a predicted time.
- Store successful prediction inputs and outputs as MongoDB documents.
- Demonstrate the role of NoSQL in a small end-to-end intelligent application.

4. Technologies Used

Technology	Purpose
MongoDB Atlas	Cloud-hosted NoSQL document database used to store delivery records and prediction history.
PyMongo	Python driver used to connect to MongoDB, select databases and collections, and insert/read documents.
Python	Main programming language for data processing, model training and application logic.
Pandas and NumPy	Data loading, cleaning and tabular transformation.
Scikit-learn	Random Forest Regression, train/test split and evaluation metrics.
Streamlit	Web interface for entering inputs and viewing predicted delivery time.
Jupyter Notebook	Interactive environment used for exploration, preprocessing, training and evaluation.
Pickle	Serialization of the trained model and the expected model-column list for reuse by the app.

5. Dataset Description

The project uses a CSV dataset named Food_Delivery_Times.csv. The observed dataset contains 1,000 rows and 9 columns. Each row represents a delivery record. The fields used for model development are summarized below.

Column	Description
Order_ID	Identifier for an order; not used as a predictive feature.
Distance_km	Delivery distance in kilometres.
Weather	Weather condition associated with the delivery.
Traffic_Level	Traffic category during the delivery.
Time_of_Day	Time-of-day category.
Vehicle_Type	Vehicle used for the delivery.
Preparation_Time_min	Food preparation duration in minutes.
Courier_Experience_yrs	Courier experience in years.
Delivery_Time_min	Observed delivery duration in minutes; prediction target.

Missing values were observed in Weather, Traffic_Level, Time_of_Day and Courier_Experience_yrs (30 missing values in each of these fields in the loaded dataset). The preprocessing stage fills categorical missing values using the mode and fills missing courier experience values using the median. Order_ID is an identifier and is not treated as a numerical predictor.

6. Intelligent NoSQL Database Design

6.1 Database and collections

The MongoDB database used by the application is FoodDeliveryDB. It contains two logical collections:

- deliveries: stores the delivery dataset as individual documents for retrieval and data access.
- prediction_history: stores the user-provided input values, the model's predicted delivery time and a creation timestamp.

6.2 Document-oriented data model

MongoDB stores records as BSON documents, which have a JSON-like structure. This suits the project because a delivery record consists of named fields and may include both numerical and categorical values. The application can store and retrieve a document without defining a fixed relational table schema first.

Illustrative delivery document (example structure):

```
{
 "Order_ID": "example-order",
 "Distance_km": 5.2,
 "Weather": "Clear",
 "Traffic_Level": "Medium",
 "Time_of_Day": "Evening",
 "Vehicle_Type": "Bike",
 "Preparation_Time_min": 18,
 "Courier_Experience_yrs": 2.5,
 "Delivery_Time_min": 36.0
}
```

The values above are illustrative examples of document structure, not a claim that this exact record exists in the database.

6.3 PyMongo operations

PyMongo is used from Python to create a MongoClient connection, select FoodDeliveryDB, access the deliveries collection, insert dataset records and retrieve documents for processing. The Streamlit application uses the prediction_history collection to insert prediction records. The application checks database connectivity before attempting to save a prediction. Credentials should be stored outside source code, such as in an environment variable, and must not be included in a submitted ZIP file.

7. System Architecture and Workflow

1. Dataset ingestion: read the CSV file with Pandas and insert delivery records into MongoDB using PyMongo.
2. Data retrieval: query the deliveries collection and convert the returned documents into a tabular DataFrame.
3. Preprocessing: impute missing categorical values with the mode, impute missing courier experience with the median, and one-hot encode categorical features.
4. Model training: split the processed data into training and test sets (80%/20%, using random_state=42) and fit a Random Forest Regressor with 100 trees.
5. Evaluation: generate test-set predictions and calculate MAE and R-squared.
6. Model persistence: save the trained model to delivery_model.pkl and the expected feature-column list to model_columns.pkl.
7. Prediction: Streamlit collects inputs, applies the same encoding and aligns columns to the saved model-column list, then obtains a delivery-time estimate.
8. History storage: the application stores the input values, predicted time and timestamp in MongoDB's prediction_history collection.

8. Machine Learning Methodology

8.1 Target and input features

The target variable is Delivery_Time_min. The model uses Distance_km, Preparation_Time_min, Courier_Experience_yrs and encoded representations of Weather, Traffic_Level, Time_of_Day and Vehicle_Type. The order identifier is excluded because it identifies a record rather than describing delivery conditions.

8.2 Preprocessing

- Categorical missing values are replaced with the most frequent category (mode).
- Missing courier-experience values are replaced with the median.
- Categorical variables are transformed using one-hot encoding.
- The application aligns incoming prediction features with the saved model-column list and fills absent encoded columns with zero, ensuring that the feature layout matches the model's training layout.

8.3 Model selection

A Random Forest Regressor was selected. It combines predictions from multiple decision trees and can model non-linear relationships between delivery conditions and delivery duration. The model was configured with 100 estimators. The available experiment used an 80/20 train-test split and random_state=42 to make the split reproducible.

8.4 Evaluation metrics

Mean Absolute Error (MAE) measures the average absolute difference between actual and predicted delivery times, in minutes. R-squared (R^2) indicates the proportion of target variance explained by the model relative to a baseline based on the mean.

Metric	Observed result	Interpretation
Mean Absolute Error (MAE)	6.85 minutes	Predictions differ from actual values by about 6.85 minutes on average in absolute terms.
R-squared (R^2)	0.791	The model explains about 79.1% of the variance in the test target for this experiment.

These metrics describe the recorded test split and do not guarantee the same performance on future orders. Evaluation on a larger, newer and more representative dataset would be needed before operational deployment.

9. Streamlit Application

The Streamlit interface provides input controls for distance, preparation time, courier experience, weather, traffic level, time of day and vehicle type. When the user selects the prediction button, the app creates a single-row DataFrame, applies one-hot encoding, aligns the columns to the model's saved feature list and calls the trained model. The predicted delivery time is displayed in minutes.

After a prediction, the app attempts to save the entered values, predicted duration and timestamp to MongoDB's prediction_history collection. This makes the database useful during application operation, not only during model training.

10. NoSQL Concepts Demonstrated

- Document model: each delivery and prediction is represented as a document with named fields.
- Flexible schema: documents can hold mixed types such as numbers, strings and timestamps, and fields can evolve as requirements change.
- Collections and database organization: related documents are grouped into deliveries and prediction_history collections within FoodDeliveryDB.
- Python-driver integration: PyMongo provides database connectivity and CRUD-style operations from Python.
- Persistence of application events: prediction results are stored so that previous estimates can be reviewed or analyzed later.
- Separation of concerns: MongoDB handles document persistence, the ML model handles prediction, and Streamlit provides the user interface.

11. Testing and Results

Test	Outcome
Database connectivity	MongoDB Atlas connection was tested through

	PyMongo.
Dataset storage and retrieval	Delivery records were inserted into and retrieved from the deliveries collection.
Preprocessing and training	The dataset was cleaned and used to train the Random Forest Regression model.
Model evaluation	The recorded evaluation produced MAE = 6.85 minutes and $R^2 = 0.791$.
Web prediction	The Streamlit application accepted order details and displayed an estimated delivery time.
Prediction history	Prediction records were checked in MongoDB after using the application.

12. Limitations

- The dataset contains 1,000 records, which may not represent all regions, restaurants, traffic conditions or delivery fleets.
- Model accuracy depends on the quality and representativeness of the available data.
- The app's prediction quality may be affected by differences between training data and real-world conditions.
- The current implementation is a prototype and does not include authentication, role-based access or a production monitoring system.
- The report describes the implemented prototype; database credentials must be configured securely outside the submitted source code.

13. Future Enhancements

- Collect more real-world delivery records and retrain the model periodically.
- Compare Random Forest with Gradient Boosting, XGBoost and other regression algorithms.
- Create indexes on commonly queried prediction-history fields such as Created_At, if history queries grow.
- Add filters and a dashboard to analyze saved predictions by date, traffic or vehicle type.
- Add input validation, user authentication and improved error handling.
- Move the MongoDB connection URI to an environment variable and provide a safe .env.example file without secrets.
- Deploy the application and configure database access using least-privilege credentials.

14. Conclusion

The Food Delivery Time Prediction System demonstrates an end-to-end intelligent application that combines a NoSQL document database, a Python machine-learning workflow and a web interface. MongoDB Atlas stores delivery records and prediction history, while PyMongo connects the Python application to the database. A Random Forest Regression model predicts delivery duration from order and delivery conditions. The recorded experiment achieved an MAE of 6.85 minutes and an R^2 of

0.791. The project therefore illustrates how MongoDB can support a practical data-driven application, with machine learning providing the prediction capability and Streamlit making it accessible to users.

References

- MongoDB Documentation. MongoDB Manual. https://www.mongodb.com/docs/manual/
- PyMongo Documentation. https://pymongo.readthedocs.io/
- Scikit-learn Documentation. RandomForestRegressor. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html
- Streamlit Documentation. https://docs.streamlit.io/
- Pandas Documentation. https://pandas.pydata.org/docs/

Appendix A: Project Files

- Food_Delivery_Prediction.ipynb — notebook containing exploration, preprocessing, model training and evaluation.
- app.py — Streamlit application code.
- delivery_model.pkl — serialized trained model.
- model_columns.pkl — saved list of model input columns.
- Food_Delivery_Times.csv — source dataset.